

用 AI Agent 自动构造 Type Denotation / Logical Relation

研究计划：Benchmark 构建（与南京大学张子豪合作）与 Solution 探索

周喆（Zhe Zhou）

2026 年 8 月

摘要

本项目研究如何让 AI Agent 自动构造 PL / 程序验证论文中最核心的语义不变量（type denotation、term denotation、logical relation、semantic domain），把 AI for Proof 从“写 tactic”推进到“发现证明的核心不变量”。本文档把工作拆成两部分：**Benchmark**（近期，由南京大学张子豪同学负责）：构建“给定语言与定理、构造语义不变量”的任务集，从类型系统推广到 abstract machine / definitional interpreter / staged evaluation 等更广义的 PL 语义工作；**Solution**（后续，周喆负责）：agent 系统原型、证明失败反馈回路、以及“哪些复杂性来自问题本身、哪些来自证明框架”的比较研究。未来该方向天然连接 logical relation、denotational semantics、substructural logic 与 separation logic，可作为与北京大学数学科学学院合作的切入点。

目录

1	核心理念与定位	2
2	背景与动机	3
2.1	为什么 type denotation / logical relation 是核心难点	3
2.2	与 AI for Proof 现状的差距	3
2.3	一个被忽视的事实：大量 PL 语义工作没有机械化形式化	3
3	Benchmark：“语义不变量构造”任务集（张子豪）	4
3.1	统一任务格式（问题卡片）	4
3.2	三类任务（类型系统层）	4
3.3	推广：超越类型系统（abstract machine 线）	5
3.4	任务优先级与难度分级	6
3.5	交付物与里程碑	6
3.6	工具链建议	7

1 核心想法与定位	2
4 Solution: Agent 系统与科学问题 (后续探索)	7
4.1 系统 workflow	7
4.2 关键技术挑战	8
4.3 科学问题: 证明框架负担与 denotation 发现	8
4.4 为什么这个问题重要	9
5 路线图与分工	9
5.1 与北京大学数学科学学院的合作	9
5.2 立即可以开始的事情	10

1 核心想法与定位

本项目 (暂名 “AI Denotation Agent”) 的目标是: 让 *AI Agent* 自动构造 *PL* / 程序验证论文中最关键的语义不变量, 即 *type denotation*、*term denotation*、*logical relation*, 以及在必要时选择合适的 *semantic domain*。

目标不是让 LLM 简单地把 *proof script* 补完, 而是让它参与 “为什么这个类型系统是 *sound* 的” 这一最核心的设计层面: 在给定语言语法、操作语义、*typing rules* 与目标 *soundness theorem* 之后, 自动提出候选的 *denotation* / *logical relation*, 并迭代地对每条 *typing rule* 尝试语义 *soundness* 证明, 在失败时分析原因 (不变量太强、太弱、缺少 *resource*、缺少 *monotonicity*、*semantic domain* 不够) 并修改设计。

关键推广: 不局限于类型系统。 “构造正确的不变量” 这一核心任务并不只出现在类型系统里。抽象机 (*abstract machine*)、定义式解释器 (*definitional interpreter*)、程序变换 (例如 *staged evaluation* 中的 *let-insertion*) 等一大类 *PL* 语义工作, 其正确性 (*soundness*、语义等价、语义保持) 同样依赖一个精心选择的语义结构, 而且这些工作大多只有非形式化论证, 甚至完全没有机械化形式化 (详见 §3)。因此本项目把 *BENCHMARK* 从 “类型系统” 推广到 “一般 *PL* 语义工作”: 既扩大了数据规模, 也使 “语义不变量构造” 成为一个真正通用的 *AI for Proof* 任务。

近期分工。

- 张子豪 (南京大学): 负责 *BENCHMARK* 构建与基线评估 (§3)。这部分产出明确、可以立即启动: 编写问题卡片、运行现有 LLM agent 作为基线、记录并整理结果。
- 周喆: 负责整体设计与 *SOLUTION* 探索 (§4): agent 系统原型、失败反馈回路、证明框架负担的比较研究。

远期合作。 该项目天然连接 *logical relation*、*denotational semantics*、*substructural logic*、*separation logic* 等数学与逻辑内容, 未来希望邀请北京大学数学科学学院的同事加入 (详见 §5.1), 形成 “数院提供数学语义基础、系统 / *PL* 方向提供语言与验证问题、*AI agent* 提供自动化” 的合作闭环。

2 背景与动机

2.1 为什么 type denotation / logical relation 是核心难点

在 PL 与程序验证论文中，有一大类工作是在设计类型系统（例如 Rust 的 ownership / borrowing 保证内存安全）。对简单语言，type soundness 通常可以用 progress + preservation 证明 [19]；但对更复杂的类型系统——Rust 的 lifetime、unsafe 抽象、并发、substructural resource 等——纯语法证明往往不够，通常需要 denotational semantics 或 logical relation。

这类证明的核心难点，是如何定义合适的 *type denotation / logical relation*。它本质上就是程序验证中的 inductive invariant：

- 定义太弱，推不出安全性；
- 定义太强，typing rule 无法证明 sound；
- 定义方式不合适，证明会非常复杂，甚至需要引入新的 semantic domain（separation logic、resource algebra、lifetime logic、step-indexed Kripke logical relation 等）。

两篇背景文献：

- Timany et al. [14]：说明为什么 semantic / logical type soundness 比 progress + preservation 更强，特别适合解释 abstraction 与 unsafe / safe 交互；
- Reynolds [11]：提供 intrinsic / extrinsic semantics 的经典视角——把类型看成程序语义上的 property / relation，而非只看 typing derivation 本身。

2.2 与 AI for Proof 现状的差距

目前很多 AI for Proof 工作集中在：证明单个 lemma、生成 tactic、proof repair、theorem proving benchmark。但在 PL / 程序验证里，真正困难的地方经常不是某一步 tactic，而是：

我们到底应该定义什么样的 *invariant*，才能让整个 *soundness proof* 成立？

这个问题目前基本只能靠人的洞察力解决。如果能被 agent 部分自动化，将直接降低 PL mechanization 的门槛，并让研究者更快地验证一个新的 typing rule 是否 sound、探索 alternative denotation。

2.3 一个被忽视的事实：大量 PL 语义工作没有机械化形式化

类型系统之外，还有一大类 PL 语义工作——abstract machine、definitional interpreter、staged evaluator 等——它们论文里的正确性声明（soundness、语义等价、语义保持）通常只有非形式化论证，没有公开的机械化形式化（详见 §3.3）。这些工作恰好是“不变量构造”任务最干净、也最有补全价值的数据来源。

3 Benchmark: “语义不变量构造”任务集（张子豪）

这一部分是可以立即启动的工作，由张子豪同学负责执行，周喆提供设计与工具链支持。BENCHMARK 的目标是回答一个定量问题：给定语言定义、语义与目标定理，现有的 *LLM agent* 能在多大程度上自动构造出正确、可证明的语义不变量？

3.1 统一任务格式（问题卡片）

每个任务用一张问题卡片（problem card）描述，尽量机器可读、可复用：

输入：

- 语言语法（BNF，或已有的 Coq / Lean 定义，优先复用现有 artifact）；
- 操作语义（big-step / small-step / abstract machine / definitional interpreter）；
- typing rules，或待验证的目标语义对象；
- 目标定理（type soundness / 抽象解释 soundness / 变换语义保持 / 机器与解释器等价）；
- 可选：例子、测试、已有 proof attempt。

期望输出：

- semantic domain（或 abstract domain + Galois connection / abstraction relation）；
- type denotation / term denotation / logical relation（或 simulation relation）；
- context interpretation；
- 每条 typing rule（或每个语义步骤）对应的 proof obligation；
- 完整证明脚本（Rocq / Lean），要求无 **Admitted**。

3.2 三类任务（类型系统层）

类型系统层的任务分为三类，分别对应不同研究目的：

1. **复现与简化**：论文本身已有 denotation / logical relation。例如 RustBelt [8]、RustHornBelt [10]、VetusBelt [5]，以及 reachability types 的逻辑关系工作 [2]——这些工作的 type denotation / logical relation 本身就是核心贡献。这里的目标不是假装从零发明，而是看 agent 能否根据 typing rules 与 soundness theorem 找到类似的不变量，或者发现更简单、更模块化的定义方式。
2. **证明结构迁移**：论文有 soundness proof，但不是 denotation / logical relation（多为 progress + preservation，如 STLC、System F、Featherweight Java [6]）。目标：验证 agent 能否把 syntactic proof 问题重铸成 semantic type soundness / logical relation 问题，而不是只会复述原论文。

3. **补全缺失证明**：系统没有完整证明，或只有非形式化论证。输入只有语言定义、typing rules、例子与目标定理。成功则补全 soundness proof；失败也可能暴露 typing rule 本身不 sound，或指出缺少必要的语义结构。这类任务最接近真实研究场景。

3.3 推广：超越类型系统（abstract machine 线）

原想法只覆盖“类型系统”；本项目的关键推广是：把“构造语义不变量”任务从类型系统推广到一般的 PL 语义工作。理由有二：(1) 任务结构同构——都是“给定形式定义与目标定理，构造让证明成立的语义结构”；(2) 数据更丰富——很多这类工作没有机械化形式化，benchmark 直接产生研究价值。

表 1: abstract machine 线候选任务（正确性声明多为非形式化论证，未见公开机械化形式化）

论文	正确性声明	要构造的语义结构
Van Horn & Might, <i>Abstracting Abstract Machines</i> (ICFP'10) [15]	抽象机上的抽象解释（CFA）相 对于具体机器语义是 sound 的	abstract domain + abstraction / simulation relation
Johnson & Van Horn, <i>Abstracting Definitional Interpreters</i> (ICFP'14) [7]	把抽象机统一为 definitional interpreter 后，抽象版本仍 sound	monadic abstract interpreter 的 soundness invariant
Danvy & Nielsen, <i>Refocusing in Reduction Semantics</i> (RS-04-26, 2004) [4]	refocus 与 plug-then-decompose 计算相同结果	约简语义与抽象机之 间的等价 / simulation
Danvy & Millikin, <i>Refunctionalization at Work</i> (SCP'09) [3]	defunctionalization 的左逆在语 义上保持计算	函数表示与一等函数 间的语义等价
Wei, Decker & Rompf, <i>Refunctionalization of Abstract Abstract Machines</i> (ICFP'18, pearl) [16]	AAM 与 ADI 两个框架通过 refunctionalization 相互转换	机器状态与解释器状 态之间的双向转换保 持 soundness
Wei, Tan & Zhong, <i>Let It Be Optimized: Multi-Stage Evaluators with Let-Insertion</i> (ICFP'26, pearl) [18]	staged evaluator 中 let-insertion 及各步优化均保持程序语义	分阶段求值 / let-insertion 的语义保 持不变量

这些任务的“语义不变量”与类型系统的 logical relation 一一对应：类型系统对应 type denotation / logical relation；机器与解释器对应 machine state 之间的 (bi-) simulation；变换正确性对应 refinement / equational theory。Wei et al. [18] 是该线最近的工作，其 related work 展示了完整脉络（MetaML [13]、LMS [12]、staged abstract interpreters [17] 等），可以作为问题卡片书写的文献地图。

说明 3.1. “没有机械化形式化”这一判断需要在构建问题卡片时逐篇核实（搜索 artifact、公开 Coq/Lean 代码）。如果某篇已有形式化，就把它降级为第一类（复现与简化）任务——这本身也是 benchmark 元数据的一部分。

3.4 任务优先级与难度分级

表 2: benchmark 优先级（P0 最先启动）

优先级	任务	说明	类别
P0	STLC logical relation (call-by-value)	校准任务：有大量现成 Coq/Lean 参考，用于验证 agent 管线与评价指标	一
P0	System F parametricity	校准任务：逻辑等价 + 自由定理，验证 agent 能否处理高阶量词	一
P1	RustBelt 最小片段 (λ_{rust} 子集, own/shr 模型)	已有 Iris + Coq 形式化，测试 agent 能否恢复 own/shr split、current/future model	一
P1	Reachability Types 片段	OOPSLA’25 已给出 logical relation [2], POPL’24 有 Coq 机械化 [1]; 测试简化 / 重组织	一
P2	Featherweight Java 重铸	把 syntactic type soundness 重铸为 semantic soundness [6]	二
P2	region / ownership 小语言	测试 agent 能否发现资源敏感 (resource-sensitive) 语义	二 / 三
P3	AAM / ADI 的 soundness 形式化	补全缺失证明，见 表 1 前两行	三
P3	Refunctionalization AAM $\leftrightarrow$ ADI 等价性	补全缺失证明，见 表 1 第五行	三
P3	let-insertion 语义保持 (ICFP’26 pearl)	最新、难度最高，见 表 1 末行	三

3.5 交付物与里程碑

每个 case 的交付物（四件套）：

1. **问题卡片**：输入 / 输出 / 目标定理，机器可读（Markdown 或 JSON），附语言定义骨架（Rocq / Lean）；
2. **运行日志**：agent 的 candidate denotation、失败尝试与原因分析、最终结果（成功的定义与证明）；

3. 形式化产物：Rocq / Lean 证明脚本（无 Admitted）+ 说明文档；
4. 元数据：难度、是否已有公开形式化、参考文献、备注（例如发现原规则过强/过弱）。

表 3: 建议里程碑

时间	里程碑	内容
第 1–2 周	M0: 跑通管线	完成 P0 两个校准 case 的问题卡片；用现有 LLM agent（如 Codex）+ Rocq 跑出基线；确定运行记录模板与评价指标
第 3–6 周	M1: 类型系统层	完成 P1、P2 共 4–6 个 case；整理“复现 / 迁移 / 补全”三类结果对比
第 7–12 周	M2: 推广层	完成 abstract machine 线 2–3 个 case（建议先 AAM，再 refunctionalization，最后 let-insertion）；产出中期报告
持续	元数据维护	每篇候选论文核实是否有公开形式化，维护 benchmark 清单

3.6 工具链建议

- 证明助手：Rocq 9.1 + stdpp（团队已有成熟经验，见 UnderLogicAndType 项目）；Lean 4 作为对照后端，方便与“证明自动化”生态对接；
- 框架：Iris / separation logic 仅在对齐原论文时使用（P1 的 RustBelt 片段）；其他 case 鼓励“直接定义”的轻量路线，以便比较框架负担 (§4.3)；
- 自动化：coqhammer、lia、eauto、QuickChick（反例搜索）等；
- Agent 基线：Codex / Claude 等现成 agent，统一 prompt 模板，记录每次运行。

4 Solution: Agent 系统与科学问题（后续探索）

BENCHMARK 为 SOLUTION 提供数据与评价基准；SOLUTION 反过来决定 BENCHMARK 的问题卡片该怎么出。两者并行推进：BENCHMARK 先跑基线，SOLUTION 在此之上迭代 agent 能力。

4.1 系统 workflow

收敛之后，进一步检查：某个 denotation 成功证明了原 typing rules，是否还允许更强、更自由的 typing rules？如果是，说明原论文的规则可能过于保守。这个方向不仅能复现已有证明，还可能发现已有类型系统设计中的限制或改进空间。

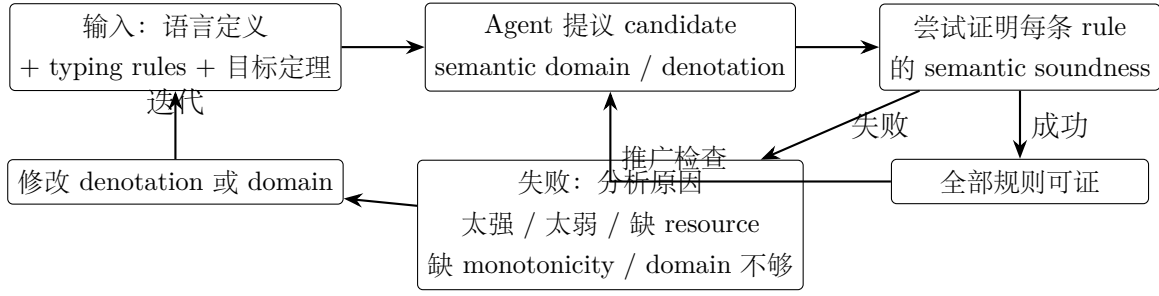


图 1: Agent 迭代 workflow: 构造 denotation 的过程类似程序验证中寻找 inductive invariant。

4.2 关键技术挑战

- **候选生成**: 如何让 LLM 生成“有前途”的 denotation? 把 denotation 的常见结构 (product / sum / function / existential / step-indexing / resource algebra / Kripke world) 做成一个小的组合 DSL, 让 agent 在此空间内搜索, 而不是自由文本生成;
- **失败反馈回路**: 把证明失败 (未闭合目标、循环依赖、不可判定的子目标) 转成可操作的信号; 用 QuickChick / small-model 搜索找反例, 区分“denotation 错了”与“证明还差一步”;
- **证明后端**: Rocq / Lean 双后端; 证明自动化 (coqhammer、lia、eauto) 与 denotation 生成的解耦, 使失败分析聚焦于不变量设计而非 tactic 细节;
- **评价指标**: 成功率之外, 还要度量 denotation 的可读性 / 模块化、是否需要引入新的 semantic domain、能否发现更强的 typing rules。

4.3 科学问题: 证明框架负担与 denotation 发现

很多已有证明依赖大型框架 (如 Iris [9]、separation logic 库、step-indexed LR 库)。这些框架表达力强, 但也可能带来额外负担: 某些 denotation 必须绕着框架接口写, 某些概念要通过复杂的 resource algebra 或 proof pattern 间接表达, 导致定义不直观、proof script 很重、可读性差。

如果 agent 能在同一个 soundness 任务上尝试不同的 semantic domain、不同的 denotation factoring、不同的 proof architecture, 我们就可以系统地比较:

- 哪些复杂性来自问题本身, 哪些来自当前证明框架的限制;
- 是否存在更直接、更可读的 denotation;
- 是否需要为这类证明设计新的 library abstraction 或新的 semantic framework。

因此这个项目不只是评估 agent 能否证明定理, 也可以反过来帮助评估 Iris 等框架在表达某类 type denotation / logical relation 时的工程负担, 并为设计更好的 proof harness 提供指导。

4.4 为什么这个问题重要

1. **帮助设计复杂类型系统**: 研究者可以更快验证 typing rule 是否 sound, 探索 alternative denotation;
2. **降低 PL mechanization 门槛**: 很多论文形式化证明的难点在 denotation / logical relation 设计;
3. **发现已有系统的不足**: agent 可能发现原 denotation 过于复杂、原 typing rules 过于保守, 甚至指出某些规则实际不 sound;
4. **连接数学逻辑与系统验证**: 天然结合 logical relation、denotational semantics、substructural logic、separation logic 与程序验证;
5. **RustBelt 风格 case study 有影响力**: 若能在 RustBelt 风格 fragment 上展示 agent 自动发现或简化 denotation, 会很有说服力, 也容易获得 PL 社区关注;
6. **改进现有证明框架**: 见 §4.3。

5 路线图与分工

表 4: 阶段划分与负责人

阶段	时间	内容	负责人
阶段 0	2026-08 起	BENCHMARK 构建: 问题卡片、基线运行、元数据 (§3)	张子豪 (周喆指导)
阶段 1	2026 秋	SOLUTION 原型 v1: 针对 P0/P1 的 denotation 候选生成 + 迭代证明	周喆
阶段 2	2027 春	推广到 abstract machine 线; 框架负担比较实验	张子豪 + 周喆
阶段 3	2027 年	论文产出: benchmark 报告 / 方法论文; 评估与北大数院合作	全体

5.1 与北京大学数学科学学院的合作

这个方向为数学侧提供了明确的问题来源: logical relation 的模型论基础、denotational semantics 中的 domain theory、substructural logic 与 categorical semantics、step-indexing 与 Kripke 语义。反过来, 数院可以为 denotation 的“正确形式”提供理论指导, 例如:

- denotation 的范畴 / 类型论表述 (intrinsic vs. extrinsic semantics [11]);

- 不同 resource algebra 之间能否建立系统的比较与合成；
- “最优 / 最简 denotation” 能否在某个数学意义下被刻画（例如作为某个 universal property）。

这些问题的答案，既能让 agent 的搜索更有方向，也能把“AI 自动构造不变量”做成一个有数学深度的研究方向。

5.2 立即可以开始的事情

1. 张子豪按 §3 完成 P0 两张问题卡片，并跑通一次 Codex/Rocq 基线；
2. 维护候选论文清单（类型层 + machine 层），逐篇核实是否有公开形式化；
3. 固定运行记录模板与评价指标，保证后续结果可复现、可比较。

参考文献

- [1] Yuyan Bao, Guannan Wei, Oliver Bračevac, Songlin Jia, Yuxuan Jiang, Qiyang He, and Tiark Rompf. Polymorphic reachability types: Tracking freshness, aliasing, and separation in higher-order generic programs. *Proceedings of the ACM on Programming Languages*, 8(POPL):14:1–14:32, 2024. doi: 10.1145/3632856.
- [2] Yuyan Bao, Songlin Jia, Guannan Wei, Oliver Bračevac, and Tiark Rompf. Modeling reachability types with logical relations: Semantic type soundness, termination, effect safety, and equational theory. *Proceedings of the ACM on Programming Languages*, 9(OOPSLA), 2025.
- [3] Olivier Danvy and Kevin Millikin. Refunctionalization at work. *Science of Computer Programming*, 74(8):534–549, 2009. doi: 10.1016/j.scico.2007.10.007.
- [4] Olivier Danvy and Lasse R. Nielsen. Refocusing in reduction semantics. Technical Report RS-04-26, BRICS, University of Aarhus, 2004.
- [5] Travis Hance, Laila Elbeheiry, Yusuke Matsushita, and Derek Dreyer. Verusbelt: A semantic foundation for verus’s proof-oriented extensions to the rust type system. *Proceedings of the ACM on Programming Languages*, 10(PLDI), 2026. doi: 10.1145/3808325.
- [6] Atsushi Igarashi, Benjamin C. Pierce, and Philip Wadler. Featherweight java: A minimal core calculus for java and GJ. *ACM Transactions on Programming Languages and Systems*, 23(3):396–450, 2001. doi: 10.1145/503502.503505.
- [7] J. Ian Johnson and David Van Horn. Abstracting definitional interpreters. In *Proceedings of the 19th ACM SIGPLAN International Conference on Functional Programming (ICFP 2014)*, pages 207–217, 2014. doi: 10.1145/2628136.2628139.

- [8] Ralf Jung, Jacques-Henri Jourdan, Robbert Krebbers, and Derek Dreyer. Rustbelt: Securing the foundations of the rust programming language. *Proceedings of the ACM on Programming Languages*, 2(POPL):66:1–66:34, 2018. doi: 10.1145/3158154.
- [9] Ralf Jung, Robbert Krebbers, Jacques-Henri Jourdan, Aleš Bizjak, Lars Birkedal, and Derek Dreyer. Iris from the ground up: A modular foundation for higher-order concurrent separation logic. *Journal of Functional Programming*, 28:e20, 2018. doi: 10.1017/S0956796818000151.
- [10] Yusuke Matsushita, Xavier Denis, Jacques-Henri Jourdan, and Derek Dreyer. Rusthornbelt: A semantic foundation for functional verification of rust programs with unsafe code. *Proceedings of the ACM on Programming Languages*, 6(POPL):44:1–44:34, 2022. doi: 10.1145/3519939.3523704.
- [11] John C. Reynolds. The meaning of types: From intrinsic to extrinsic semantics. Technical Report RS-00-32, BRICS, University of Aarhus, 2000.
- [12] Tiark Rompf and Martin Odersky. Lightweight modular staging: A pragmatic approach to runtime code generation and compiled DSLs. In *Proceedings of the 9th International Conference on Generative Programming and Component Engineering (GPCE 2010)*, pages 127–136, 2010. doi: 10.1145/1868294.1868295.
- [13] Walid Taha and Tim Sheard. MetaML and multi-stage programming with explicit annotations. *Theoretical Computer Science*, 248(1–2):211–242, 2000. doi: 10.1016/S0304-3975(00)00053-0.
- [14] Amin Timany, Robbert Krebbers, Derek Dreyer, and Lars Birkedal. A logical approach to type soundness. *Journal of the ACM*, 71(2):8:1–8:67, 2024. doi: 10.1145/3651128.
- [15] David Van Horn and Matthew Might. Abstracting abstract machines. In *Proceedings of the 15th ACM SIGPLAN International Conference on Functional Programming (ICFP 2010)*, pages 51–62, 2010. doi: 10.1145/1863543.1863553.
- [16] Guannan Wei, James M. Decker, and Tiark Rompf. Refunctionalization of abstract abstract machines: Bridging the gap between abstract abstract machines and abstract definitional interpreters (functional pearl). *Proceedings of the ACM on Programming Languages*, 2(ICFP):105:1–105:28, 2018. doi: 10.1145/3236800.
- [17] Guannan Wei, Yuxuan Chen, and Tiark Rompf. Staged abstract interpreters: Fast and modular whole-program analysis via meta-programming. *Proceedings of the ACM on Programming Languages*, 3(OOPSLA):126:1–126:32, 2019. doi: 10.1145/3360545.
- [18] Guannan Wei, Jun Tan, and Dinghong Zhong. Let it be optimized: Building multi-stage evaluators with let-insertion and optimizations in small pieces (functional pearl). *Proceedings of the ACM on Programming Languages*, 10(ICFP), 2026.

- [19] Andrew K. Wright and Matthias Felleisen. A syntactic approach to type soundness. *Information and Computation*, 115(1):38–94, 1994. doi: 10.1006/inco.1994.1093.